

Wzorzec MVC w tworzeniu aplikacji internetowych

Projekty – Zasady ogólne

Celem projektu zaliczeniowego jest zweryfikowanie umiejętności praktycznych studentów oraz sprawdzenie osiągniętych przez nich kompetencji w celu realizacji pełnoprawnej aplikacji w wybranej technologii z wykorzystaniem wzorca architektonicznego MVC.

Projekt zaliczeniowy oddawany jest w formie repozytorium na platformie GitHub. Owe repozytorium powinno zawierać:

- plik README.md, a w nim:
 - tytuł i nazwę wybranego projektu (np. Książka adresowa)
 - spis treści
 - listę i krótki opis zaimplementowanych w projekcie funkcjonalności
 - instrukcje obsługi (jak uruchomić aplikację, jakie paczki należy zainstalować i w jaki sposób)
- kod źródłowy aplikacji
- plik z przykładowymi danymi wejściowymi (jeżeli projekt tego wymaga)

Ocena projektu opiera się na:

- wykonaniu dokumentacji i opisu w pliku README.md
- realizacji wymaganych funkcjonalności
- wykorzystaniu bibliotek zewnętrznych
- poprawności implementacji widoków
- poprawności wykorzystania wzorca architektonicznego MVC

Oddanie projektu spełniającego podstawowe założenia pozwala na uzyskanie oceny 3.0.

Aby uzyskać ocenę wyższą należy wykonać co najmniej dwie rzeczy z poniższych:

- Dodanie dwóch dodatkowych modeli oraz relacji pomiędzy nimi lub pomiędzy nimi a modelem głównym. Na przykład w Projekcie 1, w zaproponowanym modelu „Książka”, można zmienić pole „autor” z typu tekstowego na obiekt „Autor”. Dodatkowo można dodać model „Wypożyczenie”, który będzie wskazywał, że dana książka została wypożyczona i tym samym nie jest aktualnie dostępna. Można również obrać inny kierunek i wprowadzić modele „Osoba” oraz „Wypożyczenie”, co umożliwi implementację funkcjonalności wypożyczania książek konkretnym osobom. Możliwości rozbudowy projektu są praktycznie nieograniczone.
- Dodanie ostrylowanej tabeli lub widoku pojedynczego obiektu w sposób schludny wizualnie.
- Rozszerzenie projektu o konfigurację umożliwiającą uruchomienie aplikacji w kontenerze Docker. Celem tego zadania jest przygotowanie środowiska, które pozwala na łatwe budowanie, uruchamianie i testowanie aplikacji niezależnie od systemu operacyjnego użytkownika.

Wymagania:

- Stworzenie pliku `Dockerfile` definiującego sposób budowy obrazu aplikacji.
- Stworzenie pliku `docker-compose.yml` (opcjonalnie), który ułatwi uruchamianie aplikacji wraz z bazą danych (np. MongoDB, PostgreSQL).
- Uaktualnienie pliku `README.md` o instrukcję uruchomienia aplikacji w kontenerze.
- Dodanie przykładowych testów jednostkowych.
- Dodanie walidacji po stronie serwera oraz klienta np. sprawdzanie wymaganych pól, formatów danych (email, daty), długości tekstu itd. Pozwala to na zwiększenie bezpieczeństwa i użyteczności aplikacji.
- Zastosowanie zewnętrznego API lub integracja z inną usługą np. pobieranie danych o książkach z API Google Books, pobieranie pogody na miejsce wydarzenia, użycie mapy do lokalizacji wydarzenia itd.
- Dodanie funkcji filtrowania lub wyszukiwania np. filtrowanie książek po autorze, wyszukiwanie wydarzeń po nazwie, sortowanie treningów po intensywności itd.
- Zaimplementowanie logiki sesji użytkownika lub prostego systemu logowania. Może to być proste logowanie oparte o sesje lub token (np. JWT). Użytkownik mógłby mieć dostęp do funkcji dodawania/edycji tylko po zalogowaniu.

W przypadku, gdy aplikacja i/lub repozytorium nie będą kompletne lub nie spełnią wszystkich wymagań – ocena końcowa projektu zostanie odpowiednio obniżona.

Lista zadań projektowych

Zadanie 1 – System zarządzania cyfrową biblioteką

Struktura projektu MVC:

- Model: tytuł, autor, rok wydania
- Kontroler: obsługa żądań HTTP, interakcja z modelem, przekazywanie danych do widoku
- Widok: lista widoków, formularz dodawania i edycji

Zadanie 2 – Platforma do organizowania wydarzeń kulturalnych

Struktura projektu MVC:

- Model: nazwa, data, miejsce
- Kontroler: obsługa żądań HTTP, interakcja z modelem, przekazywanie danych do widoku
- Widok: lista widoków, formularz dodawania i edycji

Zadanie 3 – System monitorowania postępów w nauce

Struktura projektu MVC:

- Model: nazwa, poziom zaawansowania, osiągnięcia
- Kontroler: obsługa żądań HTTP, interakcja z modelem, przekazywanie danych do widoku
- Widok: lista widoków, formularz dodawania i edycji

Zadanie 4 – System zarządzania zadaniami domowymi

Struktura projektu MVC:

- Model: opis, termin wykonania, status
- Kontroler: obsługa żądań HTTP, interakcja z modelem, przekazywanie danych do widoku
- Widok: lista widoków, formularz dodawania i edycji

Zadanie 5 – System rezerwacji biletów na wydarzenia

Struktura projektu MVC:

- Model: nazwa wydarzenia, data, liczba miejsc
- Kontroler: obsługa żądań HTTP, interakcja z modelem, przekazywanie danych do widoku
- Widok: lista widoków, formularz dodawania i edycji

Zadanie 6 – System zarządzania projektami grupowymi

Struktura projektu MVC:

- Model: nazwa, opis, lista uczestników
- Kontroler: obsługa żądań HTTP, interakcja z modelem, przekazywanie danych do widoku
- Widok: lista widoków, formularz dodawania i edycji

Zadanie 7 – System zarządzania zadaniami dla zespołu programistycznego

Struktura projektu MVC:

- Model: opis, przypisany użytkownik, status
- Kontroler: obsługa żądań HTTP, interakcja z modelem, przekazywanie danych do widoku
- Widok: lista widoków, formularz dodawania i edycji

Zadanie 8 – Kolekcja ulubionych przepisów do koktajli

Struktura projektu MVC:

- Model: nazwa, składniki, instrukcje
- Kontroler: obsługa żądań HTTP, interakcja z modelem, przekazywanie danych do widoku
- Widok: lista widoków, formularz dodawania i edycji

Zadanie 9 – System organizacji treningów fitness

Struktura projektu MVC:

- Model: nazwa, rodzaj, intensywność
- Kontroler: obsługa żądań HTTP, interakcja z modelem, przekazywanie danych do widoku
- Widok: lista widoków, formularz dodawania i edycji

Zadanie 10 – System zarządzania listą zadań do przeczytania

Struktura projektu MVC:

- Model: tytuł, autor, gatunek
- Kontroler: obsługa żądań HTTP, interakcja z modelem, przekazywanie danych do widoku
- Widok: lista widoków, formularz dodawania i edycji

Zadanie 11 – Platforma do organizacji podróży i wyjazdów

Struktura projektu MVC:

- Model: cel podróży, data, lista rezerwacji
- Kontroler: obsługa żądań HTTP, interakcja z modelem, przekazywanie danych do widoku
- Widok: lista widoków, formularz dodawania i edycji

Zadanie 12 – Katalog kolekcji filmów

Struktura projektu MVC:

- Model: tytuł, reżyser, ocena
- Kontroler: obsługa żądań HTTP, interakcja z modelem, przekazywanie danych do widoku
- Widok: lista widoków, formularz dodawania i edycji

Zadanie 13 – System monitorowania wydatków domowych

Struktura projektu MVC:

- Model: kategoria, kwota, data
- Kontroler: obsługa żądań HTTP, interakcja z modelem, przekazywanie danych do widoku
- Widok: lista widoków, formularz dodawania i edycji

Zadanie 14 – Kolekcja ulubionych przepisów kulinarnych

Struktura projektu MVC:

- Model: nazwa, składniki, instrukcje
- Kontroler: obsługa żądań HTTP, interakcja z modelem, przekazywanie danych do widoku
- Widok: lista widoków, formularz dodawania i edycji

Zadanie 15 – System organizacji imprez i wydarzeń

Struktura projektu MVC:

- Model: nazwa, data, lista gości
- Kontroler: obsługa żądań HTTP, interakcja z modelem, przekazywanie danych do widoku
- Widok: lista widoków, formularz dodawania i edycji